

# Desafio Técnico - Backend / Plataforma

---

**Prazo:** gostaríamos da apresentação amanhã, se possível.

**Depois da entrega:** conversa de 40 minutos para você defender as decisões.

Este desafio é **deliberadamente maior do que o tempo disponível**. Não é para ser terminado, é para ser **priorizado**. Parte da avaliação é o que você escolheu deixar de fora e por quê.

## Contexto

A **FieldPro** é um marketplace que conecta operadores autônomos de campo (manutenção predial, limpeza técnica, instalação) a locais que precisam de trabalho pontual - uma **Vaga** é um bloco de trabalho com data, hora de início, duração, endereço e valor.

Números atuais da plataforma:

- 120 mil operadores cadastrados, ~40 mil ativos por mês
- 8 mil locais ativos
- ~25 mil vagas publicadas por mês, com pico forte de publicação **segunda-feira entre 8h e 10h**
- Opera em dois países, com fusos horários diferentes
- Stack atual: monólito HTTP + PostgreSQL + Redis. Existe um provider externo de push notification.

## O problema

Vagas publicadas com **menos de 24h de antecedência** ("urgentes") têm taxa de preenchimento muito baixa.

Hoje o comportamento é: ao publicar uma vaga urgente, o sistema dispara push para **todos os operadores elegíveis de uma vez**. O resultado é ruim nas duas pontas - o local não preenche, e os operadores reclamam de excesso de notificação.

Produto quer substituir isso por um **dispatch em ondas**: em vez de avisar todo mundo simultaneamente, avisar grupos progressivamente maiores, parando assim que a vaga for preenchida.

## Requisitos que recebemos do Produto

Estes são os requisitos **como foram escritos pelo time de Produto**. Não foram revisados por engenharia.

1. **Ondas por prioridade.** Onda 1: operadores favoritos daquele local. Onda 2: operadores que já trabalharam naquele local antes. Onda 3: todos os operadores elegíveis dentro de um raio da vaga. O intervalo entre uma onda e a seguinte deve ser curto.
2. **Primeiro que aceita, leva.** Nunca pode existir mais de um operador confirmado na mesma vaga.
3. **Anti-spam.** Nenhum operador pode receber mais de 3 notificações por dia.

4. **Tela "vagas perto de mim"**. Precisa responder em **p99 < 200 ms** sustentando 500 req/s no pico. Filtros: raio, data, especialidade e faixa de valor. A tela deve refletir imediatamente quando uma vaga é preenchida - o operador não pode ver e tentar aceitar algo que já foi.
  5. **O local pode editar ou cancelar a vaga a qualquer momento**, inclusive com o dispatch em andamento.
  6. **Provider de push externo**. Limite de 600 requisições por segundo, pode responder 429 ou 5xx, e chama de volta um webhook nosso informando entrega/falha de cada notificação.
- 

## O que você deve entregar

Quatro arquivos, num repositório Git (público ou zip). **Respeite os limites de tamanho indicados** - concisão faz parte da avaliação.

### 1. **DECIS0ES.md** - máximo 1200 palavras + 1 diagrama

O desenho da solução. Precisa cobrir, no mínimo:

- Modelo de dados do dispatch (tabelas/entidades e estados relevantes)
- **Como você garante o requisito 2** (nunca dois confirmados na mesma vaga)
- Como as ondas são agendadas, disparadas e interrompidas
- Como você lida com o provider externo de push
- Seu plano para o requisito 4 (latência), e o que você mediria para saber se está de pé
- Uma seção **"Fora de escopo"**: o que você não fez e por quê

Um diagrama (Mermaid, ASCII ou foto de quadro branco legível). Um só.

### 2. **PERGUNTAS-E-PREMISSAS.md** - máximo 1 página

- As perguntas que você faria ao Produto / Tech Lead antes de escrever a primeira linha
- As premissas que você assumiu para não ficar travado esperando resposta
- Para cada premissa: **o que muda no seu desenho se ela estiver errada**

### 3. **Código** - apenas um pedaço, funcionando

Implemente **somente** o núcleo da aceitação da vaga ( `POST /vagas/{id}/aceitar` ou equivalente), mais um **teste automatizado que prove** que dois operadores disparando a aceitação em paralelo não resultam em duas confirmações. O teste precisa gerar concorrência de verdade (ex.: 100 requisições simultâneas), não simulada por mock.

- Stack livre. Banco livre.
- Tem que rodar com **um comando** a partir do README ( `docker compose up + npm test` , ou equivalente).
- Não implemente o resto do sistema. Não queremos ver ondas, push, filtros geográficos ou autenticação em código.

#### 4. IA.md - todos os prompts, sem limite de tamanho

Assumimos que você vai usar IA - nós usamos, o dia inteiro. Queremos ver **como**.

- **Todos os prompts que você usou**, na ordem em que usou. Não é resumo nem seleção: é o registro completo. Export da conversa, log da ferramenta ou copia e cola, o formato não importa. Se usou mais de uma ferramenta, separe por ferramenta e diga em que etapa entrou cada uma.
  - Sobre esse registro, destaque:
    - **O que a IA propôs e você rejeitou** - o que ela sugeriu, por que você recusou, o que fez no lugar. Queremos pelo menos um caso concreto.
    - **O que você deixou passar** - o que entrou no entregável sem você revisar a fundo, e por que julgou que o risco era aceitável.
- 

### Regras

- Use qualquer ferramenta, biblioteca ou IA que você usaria num dia normal de trabalho.
- Não peça ajuda a outra pessoa - a conversa de defesa vai deixar isso evidente de qualquer forma.
- Se algo no enunciado parecer incompleto, contraditório ou impossível: **isso é informação, não erro de digitação**. Trate como você trataria numa sprint real.
- Prefira profundidade a cobertura. Um eixo bem resolvido vale mais do que seis eixos superficiais.

### Como será avaliado

Nesta ordem de peso:

1. Leitura crítica do problema e qualidade das perguntas
2. Arquitetura e modelagem
3. Escalabilidade e comportamento sob carga
4. Corretude do código de concorrência
5. Uso de IA como ferramenta de engenharia

Não avaliamos: quantidade de código, cobertura de testes do sistema inteiro, beleza de README, ou ter respondido tudo.